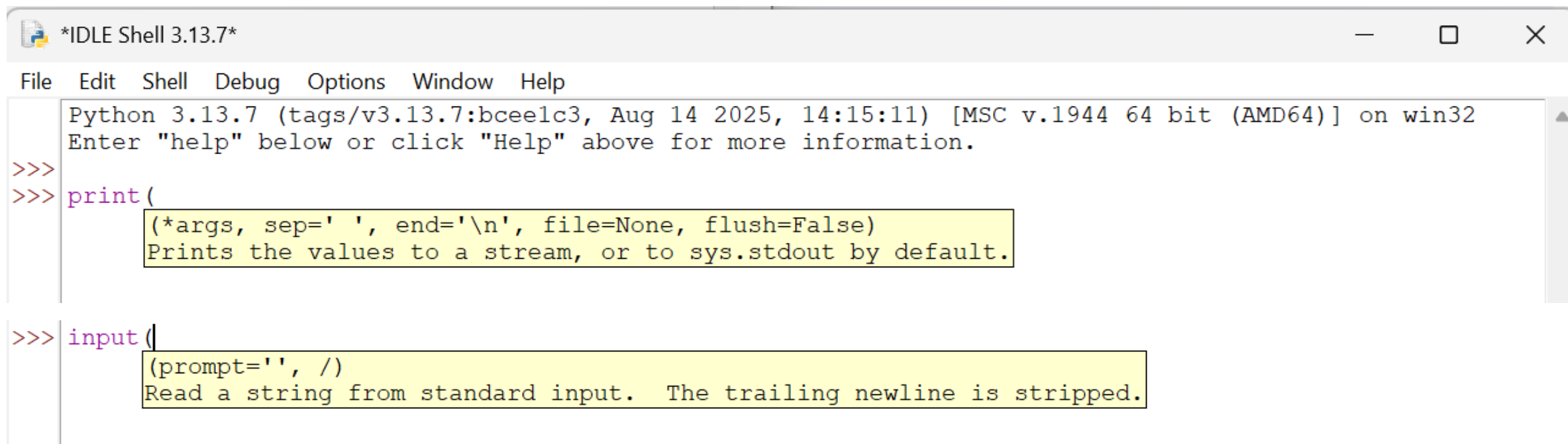


Input and Output

<https://docs.python.org/3/tutorial/inputoutput.html>



The screenshot shows the Python IDLE Shell 3.13.7 window. The title bar reads '*IDLE Shell 3.13.7*'. The menu bar includes 'File', 'Edit', 'Shell', 'Debug', 'Options', 'Window', and 'Help'. The shell displays the Python version and system information: 'Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32'. It prompts the user to enter 'help' or click 'Help'. The user has entered two commands: 'print(' and 'input(', each followed by a yellow tooltip box containing its signature and description. The 'print' tooltip shows the signature '(*args, sep=' ', end='\n', file=None, flush=False)' and the description 'Prints the values to a stream, or to sys.stdout by default.' The 'input' tooltip shows the signature '(prompt='', /)' and the description 'Read a string from standard input. The trailing newline is stripped.'

```
*IDLE Shell 3.13.7*

File Edit Shell Debug Options Window Help

Python 3.13.7 (tags/v3.13.7:bceelc3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.

>>>
>>> print(
    (*args, sep=' ', end='\n', file=None, flush=False)
    Prints the values to a stream, or to sys.stdout by default.

>>> input(
    (prompt='', /)
    Read a string from standard input. The trailing newline is stripped.
```

Zwykle wczytaniu towarzyszy konwersja z typu str na inny typ:

`int( )`, `float( )`, `complex( )`, `Decimal( )`, `Fraction( )`

oraz pomocne manipulacje funkcjami

`split( )`, `strip( )` oraz `replace( )`

Podstawowe konstrukcje języka

Instrukcja warunkowa if

```
x = 10
```

```
if x > 5:
    print("x jest większe od 5")
else:
    print("x jest mniejsze lub równe 5")
```

Pętla while

```
licznik = 0
```

```
while licznik < 5:
    print("Licznik wynosi:", licznik)
    licznik += 1
```



ZADANIE

[wyrażenie_ze_zmienną **if** warunek **else** wyrażenie_jeśli_warunek_nie_spełniony **for** zmienna **in** zbiór_lub_generator]

Utwórz listę liczb od **1** do **10**, gdzie liczby parzyste zostaną podwojone, a nieparzyste potrojone. Wydrukuj wynik, element po elemencie, ale z separatorem | pionową kreską.

Pętla for

```
owoce = ["jabłko", "banan", "wiśnia"]
```

```
for owoc in owoce:
    print(owoc)
```

```
for i in range(5):
    print("Iteracja nr", i)
```

Generatory

```
gen = (x * x for x in range(5))
```

```
for liczba in gen:
    print(liczba)
```

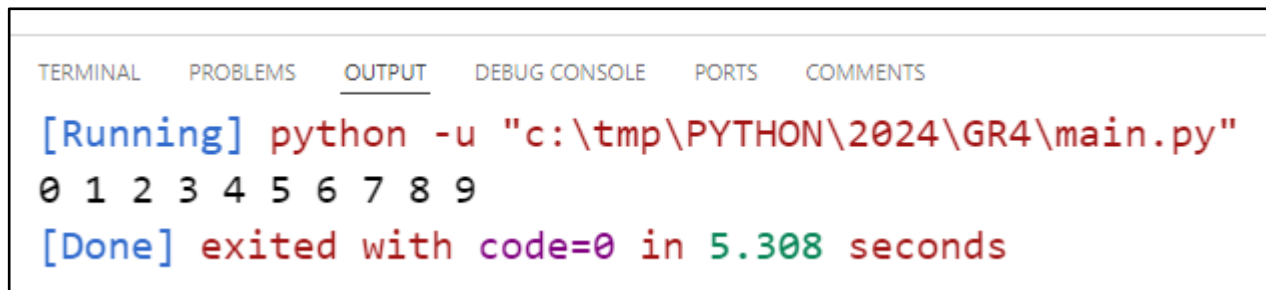
Listy składane

```
kwadraty = [x * x for x in range(5)]
print(kwadraty)
```

 ZADANIE

Zaobserwuj jak działa opcja `flush = True` oraz `flush = False` w poniższym kodzie.

```
from time import sleep
SEP = ' '
END = '\n'
FLUSH = False
for i in range(10):
    print(i, sep = SEP, end = END, flush = FLUSH)
    sleep(0.5)
```



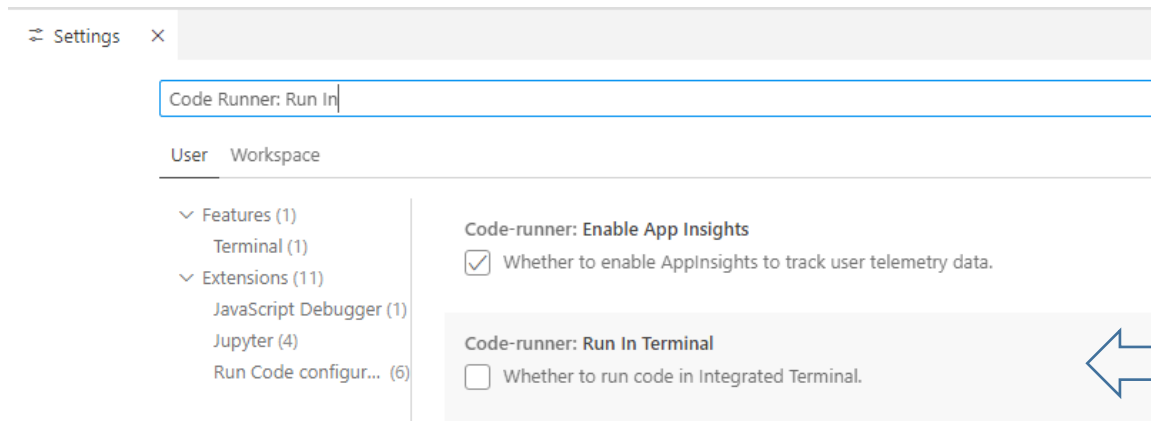
TERMINAL PROBLEMS OUTPUT DEBUG CONSOLE PORTS COMMENTS

[Running] python -u "c:\tmp\PYTHON\2024\GR4\main.py"

0 1 2 3 4 5 6 7 8 9

[Done] exited with code=0 in 5.308 seconds

Warto zauważyć, że wynikowy strumienia w panelu OUTPUT (można go otworzyć w Visual Studio Code skrótem **Ctrl+Shift U**) uruchamia program z opcją interpretera `-u` (unbuffered), czyli nadpisującą opcję `flush`. Możemy w konfiguracji ustawić uruchomienie kodu w terminalu (**Ctrl `**) w dolnej części. Wtedy również widać, że wywołanie jest np.: **python -u main.py**



 ZADANIE

Spróbuj uzupełnić poniższy kod takim wywołaniem funkcji print, żeby powstał efekt animacji złożonej z zestawu symboli, czy to kolejno (w tym samym miejscu) wyświetlanych |/-\\, czy .oOo

Podpowiedź: w pierwszym print drukujemy char, w drugim print drukujemy '\r'. W obu ustawiamy sep i end na puste ''.

```
import time
```

```
# wybierz jeden z wariantow ponizej
```

```
for char in "|/-\\"*10:
```

```
# for char in ".oOo"*10:
```

```
    print( _____ )
```

```
    time.sleep(0.2)
```

```
    print( _____ )
```

 ZADANIE

Spróbuj uzupełnić poniższy kod takim wywołaniem funkcji print, żeby powstał efekt animacji złożonej z zestawu symboli, czy to kolejno (w tym samym miejscu) wyświetlanych |/-\\, czy .oOo

```
import time

# wybierz jeden z wariantow ponizej
for char in "|/-\\"*10:
# for char in ".oOo"*10:
    print(' ', char, ' ', end='', flush=True)
    time.sleep(0.2)
    print('\r', end='', flush=True)
```

W podobny sposób można uzyskać ciekawe efekty, np. emulator pisania na maszynie, kod ASCII 9619 oznacza symbol .

```
import time

text = "Tekst pisany na maszynie!"
for char in text:
    print('\b', chr(9619), end='', sep='', flush=True)
    time.sleep(0.1)
    print('', end='\b', sep='', flush=True)
    print(char, '_', end='', sep='', flush=True)
    time.sleep(0.3)
print('\b ')
```

Przeanalizujemy hard-core'owy przykład złożenia działania różnych funkcji na łańcuchu znakowym.

wejście: `tekst = " ,,,WITAJ, ŚWIECIE!!! To jest przykładowy TEKST, który będzie OBRABIANy,,, "`

wyjście: `Będzie Jest Który Obrabiany Przykładowy Tekst To Witaj Świecie`

```
nowy_tekst = " ".join(sorted(set(slowo.strip(".,!?") for slowo in tekst.strip(" ,,").replace(",,", ",").replace("!", ".").lower().split()))).title()
```

[9] [7] [1] [2] [3] [4] [5] [10]

[8] [6]

- [1] `tekst.strip(" ,,")` usuwa spacje i przecinki z początku i końca łańcucha
- [2] `.replace(",,", ",")` zamienia podwójne przecinki na pojedyncze
- [3] `.replace("!", ".")` zamienia wykrzykniki na kropki
- [4] `.lower()` konwertuje tekst na małe litery
- [5] `.split()` dzieli tekst na listę słów
- [6] `slowo.strip(".,!?") for slowo in ...` usuwa znaki interpunkcyjne z początku i końca każdego słowa
- [7] `set(...)` usuwa duplikaty słów
- [8] `sorted(...)` sortuje słowa alfabetycznie
- [9] `" ".join()` łączy słowa w jeden łańcuch, oddzielając je spacją
- [10] `.title()` zamienia pierwszą literę każdego słowa na wielką

Tak na marginesie... przykład nieczytelnego zagnieżdżenia warunków:

```
wynik = funkcja1(x) if warunek1 else
funkcja2(x) if warunek2 else funkcja3(x)
```

- najpierw sprawdzany jest warunek1:
 - jeśli True, wykonana zostaje funkcja1(x)
 - jeśli False, sprawdzany jest warunek 2
 - jeśli True, wykonana jest funkcja2(x)
 - jeśli False, wykonana jest funkcja3(x)

 ZADANIE

Ćwiczenie na użycie funkcji `strip()`, `split()`, `replace()`, także `lower()`. Odczytaj wpisane z klawiatury zdanie, zawierające kilka słów oddzielonych przecinkami i ewentualnymi spacjami, również mające dwa przecinki obok siebie. Następnie usuń nadmiarowe spacje i znaki przed i na początku wpisywanej treści, zrób dekompozycję zawartości na słowa – umieść je w liście – ale najpierw przekonwertuj je na małe litery. Wyświetl listę i jej rozmiar.

wejście: „ , zero, jeden , dwa

wyjście: Lista słów: ['zero', 'jeden', 'dwa'] Liczba słów: 3

```
# Pobranie danych od użytkownika
```

```
wejscie = input("Podaj kilka slow oddzielonych przecinkami: ")
```

```
# ... twój kod ...
```

```
# Wyświetlenie listy słów i ich liczby
```

```
print("Lista slow:", lista_slow, "#", len(lista_slow))
```

 ZADANIE

Ćwiczenie na użycie funkcji `strip()`, `split()`, `replace()`, także `tolower()`. Odczytaj wpisane z klawiatury zdanie, zawierające kilka słów oddzielonych przecinkami i ewentualnymi spacjami, również mające dwa przecinki obok siebie. Następnie usuń nadmiarowe spacje i znaki przed i na początku wpisywanej treści, zrób dekompozycję zawartości na słowa – umieść je w liście – ale najpierw przekonwertuj je na małe litery. Wyświetl listę i jej rozmiar.

wejście: „ , zero, jeden , dwa

wyjście: Lista słów: ['zero', 'jeden', 'dwa'] Liczba słów: 3

```
# Pobranie danych od użytkownika
```

```
wejscie = input("Podaj kilka słow oddzielonych przecinkami: ")
```

```
# Usunięcie spacji z początku i końca oraz zamiana podwójnych przecinków na pojedyncze
```

```
wejscie = wejscie.strip().replace(",", " , ")
```

```
# Podział ciągu na listę słów
```

```
lista_slow = wejscie.split(" , ")
```

```
# Oczyszczenie i konwersja słów
```

```
lista_slow = [slovo.strip().lower() for slovo in lista_slow if slovo.strip()]
```

```
# Wyświetlenie listy słów i ich liczby
```

```
print("Lista słow:", lista_slow, "#", len(lista_slow))
```

kod można napisać na wiele sposobów, więc to tylko przykład ilustrujący jedną z możliwości